Ecole Polytechnique Privée de Sousse

Chapitre n°4 : Les bases de données avec sqlite3 de Python

Responsable du cours : **Dr. SGHAIER Amra**

Contact : sghaieramra1@gmail.com

Niveau : 3ème *Année Génie Informatique*
Année universitaire : 2025-2026

Sommaire

1. Introduction
2. Connexion et Curseur
3. Types de données et Contraintes
4. Création de table
5. Insertion de données
6. Sélection et récupération
7. Mise à jour - UPDATE
8. Suppression : DELETE

Qu'est-ce que SQLite ?

Définition

SQLite est un système de gestion de base de données relationnelle léger, embarqué et **sans serveur**. La base de données est simplement un **fichier** sur votre disque.

Comparaison avec d'autres SGBD

- **MySQL/PostgreSQL** : Nécessitent un serveur séparé
- **SQLite** : Simple fichier `.db`

```
# MySQL nécessite un serveur
mysql_connection = "mysql://serveur:3306/ma_base"

# SQLite = simple fichier local
sqlite_connection = "sqlite:///mon_fichier.db"
```

Pourquoi SQLite avec Python ?

Avantage	Description
Sans serveur	Pas d'installation complexe
Fichier unique	Un seul fichier `.db` facile à transporter
Intégré à Python	Module `sqlite3` dans la bibliothèque standard
Léger	Parfait pour petits projets, prototypes
Standard SQL	Supporte la plupart des commandes SQL

Quand utiliser SQLite ?

Bons cas d'usage

- Application desktop locale
- Site web à faible trafic
- Outils d'analyse de données
- Tests et développement
- Applications embarquées

Mauvais cas d'usage

- Sites web à fort trafic
- Grands volumes (> 1 To)
- Écritures concurrentes
- Accès réseau direct

Sommaire

1 Introduction
2 Connexion et Curseur
3 Types de données et Contraintes
4 Création de table
5 Insertion de données
6 Sélection et récupération
7 Mise à jour - UPDATE
8 Suppression : DELETE

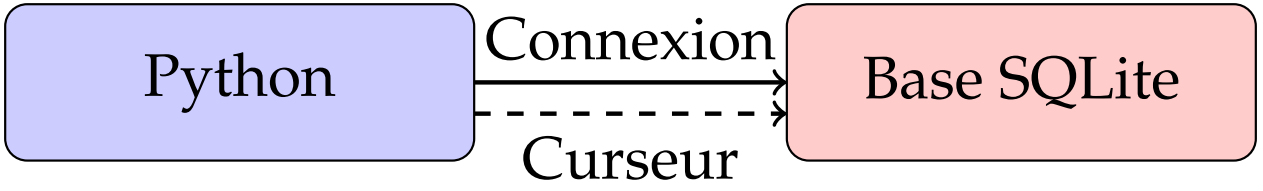
Définitions : Connexion et Curseur

Definition

Connexion : Établissement d'un canal de communication avec la base de données.

Definition

Curseur : Objet qui permet d'exécuter des requêtes SQL et de parcourir les résultats.



"La connexion est le pont entre Python et la base, le curseur est le véhicule qui traverse ce pont."

Étape 1 : Importer et se connecter

Connexion à un fichier

```
import sqlite3
# Connexion à un fichier(crée le fichier s'il n'existe
    pas)
connexion = sqlite3.connect('ma_base.db')
print(f"Base ouverte : {connexion}")
```

Étape 2 : Créer un curseur

Création du curseur

```
# Le curseur exécute les requêtes
curseur = connexion.cursor()
print(f"Curseur créé : {curseur}")
```

Étape 3 : context manager

Méthode recommandée : `with`

```
# AVEC context manager (recommandé)
# Commit automatique + fermeture automatique
with sqlite3.connect('ma_base.db') as conn:
    curseur = conn.cursor()
    curseur.execute("SELECT sqlite_version()")

    for ligne in curseur:
        print(f"Version : {ligne[0]}")

# La connexion est fermée automatiquement ici
```

Sommaire

1 Introduction
2 Connexion et Curseur
3 Types de données et Contraintes
4 Création de table
5 Insertion de données
6 Sélection et récupération
7 Mise à jour - UPDATE
8 Suppression : DELETE

Types de données SQLite

Type SQLite	Description	Exemple Python
INTEGER	Nombre entier	`42, -10, 0`
REAL	Nombre décimal	`3.14, -0.5`
TEXT	Chaîne de caractères	`"Alice"`
BLOB	Données binaires	`b'\x00\xff'`
NULL	Valeur nulle	`None`
DATE	Date	`"2024-01-15"`
BOOLEAN	Booléen (0/1)	`1 (True), 0 (False)`

Remarque

SQLite utilise le **typage dynamique** : le type est associé à la valeur, pas à la colonne.

Les contraintes (Constraints)

Contrainte	Signification
`PRIMARY KEY`	Identifiant unique, pas de NULL
`AUTOINCREMENT`	Incrémentation automatique
`NOT NULL`	Ne peut pas être vide
`UNIQUE`	Valeur unique dans la colonne
`DEFAULT valeur`	Valeur par défaut
`CHECK (condition)`	Condition de validation
`FOREIGN KEY`	Référence à une autre table

Exemple

```
age INTEGER CHECK(age > 0 AND age < 120)
email TEXT UNIQUE
est_actif BOOLEAN DEFAULT 1
```

Sommaire

1 Introduction
2 Connexion et Curseur
3 Types de données et Contraintes
4 Création de table
5 Insertion de données
6 Sélection et récupération
7 Mise à jour - UPDATE
8 Suppression : DELETE

Création d'une table - Syntaxe complète

Definition

Une **table** organise les données en lignes et colonnes. Chaque colonne a un nom, un type et éventuellement des contraintes.

```
import sqlite3
with sqlite3.connect('ecole.db') as conn:
  curseur = conn.cursor()
  curseur.execute(''' CREATE TABLE IF NOT EXISTS
      etudiants (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nom TEXT NOT NULL,
            prenom TEXT NOT NULL,
            age INTEGER CHECK(age > 0 AND age < 120),
            email TEXT UNIQUE,
            moyenne REAL DEFAULT 0.0,
            date_inscription DATE DEFAULT CURRENT_DATE,
            est_actif BOOLEAN DEFAULT 1 )''')
```

Vérification de la structure de la base

Objectif :

- Vérifier que les tables existent
- Explorer la structure d'une table
- Comprendre les métadonnées de SQLite

1) Lister les tables existantes

SQLite stocke les informations sur la base dans une table interne appelée `sqlite_master`.

```
with sqlite3.connect("ecole.db") as conn:
    for t in conn.execute(
        "SELECT name FROM sqlite_master WHERE type='
            table'"
    ):
        print("Table :", t[0])
```

Vérification de la structure de la base

2) Voir la structure d'une table

La commande PRAGMA permet d'afficher les colonnes et leurs types.

```
for col in conn.execute("PRAGMA table_info(etudiants)"):
    print(col[1], ":", col[2])    # nom : type
```

Sommaire

1 Introduction
2 Connexion et Curseur
3 Types de données et Contraintes
4 Création de table
5 Insertion de données
6 Sélection et récupération
7 Mise à jour - UPDATE
8 Suppression : DELETE

INSERT : Ajouter des données

- **INSERT** permet d'ajouter une nouvelle ligne dans une table.

```
with sqlite3.connect("ecole.db") as conn:
    conn.execute("""
        INSERT INTO etudiants (nom, prenom, age)
        VALUES (?, ?, ?)
    """, ("Martin", "Alice", 20))
```

- Les ? permettent d'éviter les erreurs et les injections SQL.

Insertion avec un dictionnaire

- **On peut utiliser un dictionnaire pour rendre la requête plus lisible.**

```
with sqlite3.connect("ecole.db") as conn:
    etudiant = {
        "nom": "Dupont",
        "prenom": "Bob",
        "age": 22 }
    conn.execute("""
        INSERT INTO etudiants (nom, prenom, age)
        VALUES (:nom, :prenom, :age)
    """, etudiant)
```

- Les :nom, :prenom... correspondent aux clés du dictionnaire

Valeurs par défaut

Insertion partielle

Si on ne fournit pas une colonne, SQLite utilise sa valeur par défaut.

```
with sqlite3.connect("ecole.db") as conn:
    # On insère seulement nom et prenom
    conn.execute("""
        INSERT INTO etudiants (nom, prenom)
        VALUES (?, ?) """, ("Durand", "Charlie"))
    # Vérification des valeurs automatiques
    for ligne in conn.execute("""
        SELECT moyenne, est_actif FROM etudiants WHERE
            prenom = ? """, ("Charlie",)):
        print("Moyenne :", ligne[0])
        print("Actif :", ligne[1])
```

- Les autres colonnes prennent automatiquement leur valeur DEFAULT.

INSERT multiple - executemany()

Definition

executemany() insère plusieurs lignes en une seule opération, bien plus efficace qu'une boucle.

```
# Liste de plusieurs étudiants
etudiants = [
    ("Dupont", "Bob", 22),
    ("Durand", "Charlie", 19),
    ("Petit", "Diana", 21)]
with sqlite3.connect("ecole.db") as conn:
    # executemany permet d'insérer plusieurs lignes
    conn.executemany("""
        INSERT INTO etudiants (nom, prenom, age) VALUES
            (?, ?, ?) """, etudiants)

print(len(etudiants), "étudiants insérés")
```

Sommaire

1 Introduction
2 Connexion et Curseur
3 Types de données et Contraintes
4 Création de table
5 Insertion de données
6 Sélection et récupération
7 Mise à jour - UPDATE
8 Suppression : DELETE

Méthodes de récupération

Definition

- **fetchone()** : Récupère la ligne suivante (une seule)
- **fetchall()** : Récupère toutes les lignes restantes
- **fetchmany(n)** : Récupère `n` lignes à la fois

fetchone() : Récupérer une seule ligne

- **fetchone()** retourne la première ligne du résultat.

```
with sqlite3.connect("ecole.db") as conn:

    resultat = conn.execute(
        "SELECT nom, prenom, age FROM etudiants WHERE id
            = ?", (1,) ).fetchone()

    if resultat:
        print("Étudiant trouvé :")
        print(resultat[1], resultat[0], "-", resultat
            [2], "ans")
    else:
        print("Aucun étudiant trouvé")
```

fetchall() : Tous les résultats

```
with sqlite3.connect("ecole.db") as conn:

    etudiants = conn.execute(
        "SELECT nom, prenom, moyenne FROM etudiants").
            fetchall()

    print("Total :", len(etudiants), "étudiants")

    total = 0
    for e in etudiants:
        print(e[1], e[0], "- Moyenne :", e[2])
        total += e[2]
    if etudiants:
        print("Moyenne générale :", total / len(
            etudiants))
```

fetchmany() : Par lots

Utile pour les gros volumes (mémoire)

```
with sqlite3.connect('ecole.db') as conn:
    curseur = conn.cursor()
    curseur.execute("SELECT nom, prenom FROM etudiants")

    lot_size = 3
    print("Récupération par lots de 3:")
    while True:
        lots = curseur.fetchmany(lot_size)
        if not lots:  # Plus de données
            break

        print(f"Lot de {len(lots)} étudiants:")
        for nom, prenom in lots:
            print(f" {prenom} {nom}")
```

Requêtes avec alias et calculs

```
with sqlite3.connect('ecole.db') as conn:
    curseur = conn.cursor()
    # Sélection avec colonnes calculées
    curseur.execute('''
        SELECT
            prenom || ' ' || nom AS nom_complet, age,
                moyenne,
            CASE
                WHEN moyenne >= 16 THEN 'Excellent'
                WHEN moyenne >= 14 THEN 'Très bien'
                WHEN moyenne >= 12 THEN 'Bien'
                WHEN moyenne >= 10 THEN 'Passable'
                ELSE 'Insuffisant'
            END AS appreciation
```

Requêtes avec alias et calculs

```
    FROM etudiants
    WHERE moyenne IS NOT NULL
    ORDER BY moyenne DESC
''')

print("Classement des étudiants :")
for nom, age, moy, app in curseur:
    print(f"  {nom:20} | {age:2} ans | {moy:5.2f} |
        {app}")
```

Sommaire

1 Introduction
2 Connexion et Curseur
3 Types de données et Contraintes
4 Création de table
5 Insertion de données
6 Sélection et récupération
7 Mise à jour - UPDATE
8 Suppression : DELETE

UPDATE - Modifier des données

Definition

UPDATE modifie des données existantes dans une table. La clause **WHERE** est cruciale pour spécifier quelles lignes modifier.

Attention

Sans `WHERE`, UPDATE modifie **TOUTES** les lignes !

UPDATE : Modifier des données

```
with sqlite3.connect('ecole.db') as conn:
    curseur = conn.cursor()
    # Vérifier la note avant
    curseur.execute('''
        SELECT moyenne FROM etudiants WHERE prenom = '
            Bob' AND nom = 'Dupont' ''')
    avant = curseur.fetchone()[0]
    print(f"Note avant: {avant}")
    # Mise à jour
    curseur.execute('''
        UPDATE etudiants
        SET moyenne = ?, nb_absences = nb_absences + 1
            WHERE prenom = ? AND nom = ?
    ''', (14.0, 'Bob', 'Dupont'))
```

Sommaire

1 Introduction
2 Connexion et Curseur
3 Types de données et Contraintes
4 Création de table
5 Insertion de données
6 Sélection et récupération
7 Mise à jour - UPDATE
8 Suppression : DELETE

DELETE : Supprimer des données

Definition

DELETE supprime des lignes d'une table.

ATTENTION

Sans WHERE, DELETE supprime TOUTES les lignes !

```
with sqlite3.connect('ecole.db') as conn:
    curseur = conn.cursor()
# Supprime les étudiants inactifs ou avec trop d'
   absences
    curseur.execute("""
        DELETE FROM etudiants WHERE est_actif = 0 OR (
            moyenne < 5 AND nb_absences > 10) """)
    print(f"{curseur.rowcount} étudiants supprimés")
 #curseur.rowcount: indique combien ont été supprimées
```